

OSCAR - Self-Hosted Quick Start

A start-to-finish guide for an outside operator who wants to run their own copy of OSCAR on their own VPS. Uses the **prebuilt public Docker image** - no source compilation, no Node setup on the host, no GitHub credentials required for the basic install.

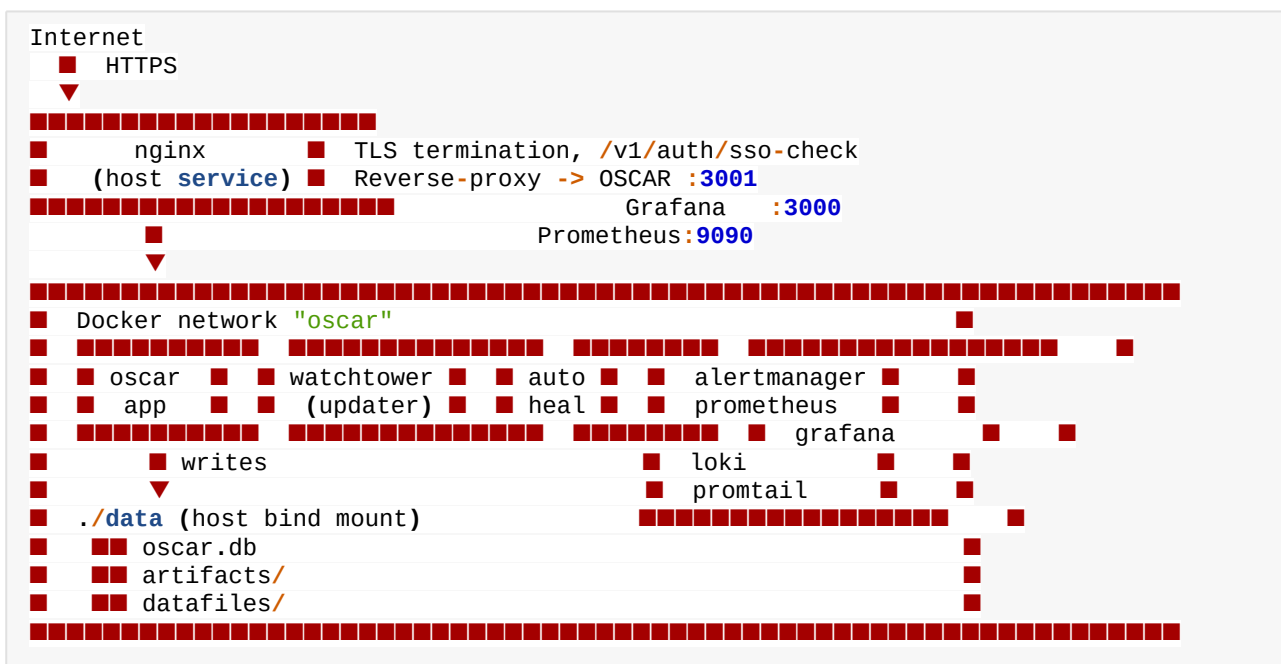
If you're a member of the OSCAR maintainer team and you're touching the canonical UIC deployment, this isn't your guide - see [OSCAR - VPS Deployment Guide.md](#) (legacy host install) and [auto-deploy-setup.md](#) (CI integration).

What you'll have running, end-to-end

After ~15 minutes you'll have:

- **OSCAR server** (Node 22, SQLite, Bruno CLI baked in) - the prebuilt [ghcr.io/top-phe/oscar-server:stable](#) image, auto-restarting via Docker
- **Watchtower** - polls GHCR every 5 min and pulls new [:stable](#) releases automatically (no SSH-and-redeploy cycle ever again)
- **Autoheal sidecar** - restarts OSCAR if its [/health](#) probe fails
- **(Optional) Prometheus + Grafana + Loki + Alertmanager** - opt-in metrics
- log aggregation + email alerts to admins

Total RAM footprint: ~250 MB without observability, ~700 MB with the full stack. Disk: ~2 GB plus your run history.



Prerequisites

VPS

Resource	Minimum	Recommended
OS	Ubuntu 22.04 / Debian 12 / any modern Linux with Docker support	Ubuntu 24.04 LTS
vCPU	2	4
RAM	2 GB	4 GB (8 GB if running observability stack)
Disk	20 GB	40 GB SSD
Public IP	required	required
Open ports inbound	22 (SSH), 80 (HTTP for Let's Encrypt), 443 (HTTPS)	same

A budget VPS plan (~5 €/month at OVH, Hetzner, Contabo, DigitalOcean basic droplet, etc.) is enough.

Software on the VPS

You need exactly four things installed on the VPS itself:

```
sudo apt update && sudo apt install -y docker.io docker-compose-plugin nginx certbot python3-certbot
```

That's it. **No Node, no npm, no Bruno on the host** - they're all inside the Docker image.

Domain name

You need a DNS record pointing to your VPS's IP. Anything works: `oscar.example.org`, `osdm.your-company.com`, etc. The rest of this guide uses `oscar.example.org` as a placeholder - substitute your own.

About SSH keys - what you actually need (and what you don't)

This trips up a lot of people because the OSCAR repo's CI workflows mention "deploy SSH keys" - those are **for the canonical UIC deployment only**, not for you.

SSH key	Who needs it	What it does
Your laptop -> your VPS	[OK] You	The standard SSH key you use to log into your own VPS (<code>ssh ubuntu@oscar.example.org</code>). If you can already SSH in, you're done.

GitHub Actions -> UIC's VPS	[X] Not you	Used by the canonical <code>top-phe/UIC_OSCAR-...</code> repo to auto-redeploy on its own production VPS after each release tag. Outsiders don't have access to that VPS and don't need this key.
GHCR pull credentials	[X] Not you	The OSCAR Docker image is on the public GitHub Container Registry - <code>docker pull ghcr.io/top-phe/oscar-server:stable</code> works without any login.

Bottom line: if you can already `ssh ubuntu@oscar.example.org` from your laptop, you have everything you need. No GitHub PAT, no deploy key, no `docker login ghcr.io`.

(If you later want to *fork* the project and run your own CI auto-deploy pipeline, you'd generate keys for that - but that's a separate, advanced topic; the basic install doesn't need it.)

Step 1 - Clone the repository

The repository is public, so this works from any machine without authentication:

```
ssh ubuntu@oscar.example.org      # your VPS
sudo mkdir -p /opt/OSCAR && sudo chown $USER /opt/OSCAR
git clone https://github.com/TOP-PHE/UIC_OSCAR-OSdm-Compliance-Automation-Runner.git /opt/OSCAR
cd /opt/OSCAR/OSCAR_Deploy
```

You only need the `OSCAR_Deploy/`, `Bruno_Collection/`, and `compatibility.json` paths - the `Oscar_Server/` source tree is irrelevant because we're using the prebuilt image. But cloning the whole repo is simpler than cherry-picking and gives you the docs locally.

Step 2 - Generate the three secrets

OSCAR needs three locally-generated secrets before first start. None of them ever leave your VPS.

```
# 1. Encryption key for credentials at rest in the SQLite DB.
# MUST be exactly 64 hex characters (32 bytes).
ENCRYPTION_KEY=$(openssl rand -hex 32) && echo "ENCRYPTION_KEY=$ENCRYPTION_KEY"

# 2. JWT signing secret for web sessions.
JWT_SECRET=$(openssl rand -hex 32) && echo "JWT_SECRET=$JWT_SECRET"

# 3. One-time bootstrap token used to create the very first admin account.
# Used exactly once, then can be unset.
PLATFORM_BOOTSTRAP_TOKEN=$(openssl rand -base64 36 | tr -d '+/') && echo "PLATFORM_BOOTSTRAP_TOKEN=$PLATFORM_BOOTSTRAP_TOKEN"
```

Save all three values to a password manager (or just to a temp text file you'll delete after Step 6 below). You'll paste them into the next step.

Why these matter: lose **ENCRYPTION_KEY** and you lose access to every stored OAuth credential (the encrypted columns become unreadable). Lose **JWT_SECRET** and every active session is invalidated on the next restart (annoying but recoverable - users just log in again). Lose **PLATFORM_BOOTSTRAP_TOKEN** after you've created the first admin and it doesn't matter - that token is single-use.

Step 3 - Configure the environment

```
cp /opt/OSCAR/OSCAR_Deploy/.env.example /opt/OSCAR/OSCAR_Deploy/.env
nano /opt/OSCAR/OSCAR_Deploy/.env      # or your editor of choice
```

Fill in **at minimum** these four values from Step 2 + your domain:

```
ENCRYPTION_KEY=<paste from step 2>
PLATFORM_BOOTSTRAP_TOKEN=<paste from step 2>
JWT_SECRET=<paste from step 2>
ALLOWED_REDIRECT_HOSTS=oscar.example.org # your domain
```

Everything else (**SMTP_***, **RUN_TIMEOUT_MS**, **MAX_CONCURRENT_RUNS**, **LOG_LEVEL**, **ALERT_RECIPIENTS**, ...) is **optional at first boot**. The **.env** file only seeds the database on the very first start - after that, all of these values are editable from the admin UI's *Server Config* tab without restarting the server. Leave the SMTP block blank for now; you'll fill it in via the UI in Step 7.

Step 4 - nginx + TLS

OSCAR doesn't terminate TLS itself - it expects an HTTPS reverse proxy in front. nginx is the canonical choice and lets you get a free Let's Encrypt certificate in 30 seconds.

4.1 - Minimal nginx site

```
sudo tee /etc/nginx/sites-available/oscar.example.org <<'EOF'
server {
    listen 80;
    server_name oscar.example.org;

    # Reverse proxy to the OSCAR container (bound to 127.0.0.1:3001 in compose)
    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_read_timeout 300;
    }
}
map $http_upgrade $connection_upgrade {
    default upgrade;
    '' close;
}
EOF
```

```
sudo ln -s /etc/nginx/sites-available/oscar.example.org /etc/nginx/sites-enabled/  
sudo nginx -t && sudo systemctl reload nginx
```

4.2 - Let's Encrypt (free TLS certificate)

```
sudo certbot --nginx -d oscar.example.org  
# Follow the prompts. Choose redirect-HTTP-to-HTTPS when asked.
```

Certbot rewrites your nginx file to add the SSL block + auto-renewal. From this point on, <https://oscar.example.org> works.

If you're enabling the optional observability stack (Step 8), you'll add three more nginx blocks for [/grafana/](#), [/prometheus/](#), [/auth/sso-check/](#). Those snippets are in [OSCAR_Deploy/nginx/oscar-metrics.conf.snippet](#). Skip for now if you're sticking to the basic install.

Step 5 - Bring the stack up

```
cd /opt/OSCAR/OSCAR_Deploy  
sudo docker compose up -d
```

What happens: - Docker pulls [ghcr.io/top-phe/oscar-server:stable](#) from the public GHCR (~150 MB) - Docker pulls [nickfedor/watchtower:latest](#) (~50 MB) - Docker pulls [willfarrell/autoheal:1.2.0](#) (~5 MB) - The [oscar](#) container starts, runs DB migrations, opens port 3001 on 127.0.0.1 (nginx is the only thing reaching it from outside) - Watchtower starts polling GHCR every 5 min - any future [:stable](#) push rolls out automatically

```
# Verify everything is healthy  
docker ps --format 'table {{.Names}}\t{{.Status}}'  
# oscar should show "(healthy)" after ~30 s; watchtower + oscar-autoheal "Up"  
  
# Check the app responds  
curl -s https://oscar.example.org/health | python3 -m json.tool  
# Should show {"status":"ok", ...}
```

Step 6 - Create the first administrator

OSCAR's web UI requires authentication, so you can't sign up via the browser yet. The bootstrap token from Step 2 unlocks one-time admin creation.

```
# Create your first admin user (replace email and password).  
# Note: the bootstrap token goes in the X-Platform-Bootstrap-Token HEADER,  
# not in the JSON body. Password policy: minimum 12 chars, must include  
# uppercase, lowercase, and a digit.  
curl -X POST https://oscar.example.org/v1/auth/bootstrap/platform-user \  
-H 'Content-Type: application/json' \  
-H 'X-Platform-Bootstrap-Token: PASTE_PLATFORM_BOOTSTRAP_TOKEN_HERE' \  
-d '{  
  "email": "you@example.org",
```

```
"password": "ChooseAStrongPasswordHere123!",
"role": "administrator"
}'
```

Expected response:

```
{ "user": { "id": "...", "email": "you@example.org", "role": "administrator" } }
```

Now sign in at <https://oscar.example.org> with the credentials you just created. You'll land on the admin welcome page with full access.

After this, **clear the bootstrap token from `.env`** so it can't be reused:

```
sudo nano /opt/OSCAR/OSCAR_Deploy/.env
# blank out the PLATFORM_BOOTSTRAP_TOKEN line
sudo docker compose restart oscar
```

Step 7 - SMTP via the admin UI (no `.env` edits needed)

This answers the "do I configure SMTP in `.env`?" question: **no, not anymore**. Since v1.7 SMTP credentials are runtime-editable from the admin UI, and since v1.9.0 they're also the single source of truth for the alerting system. One configuration, used by everything.

1. Sign in as administrator -> top nav -> **Server Config**
2. Scroll to **SMTP / Email Settings** and fill in your relay's credentials. Tested relays:

Provider	SMTP_HOST	SMTP_PORT	SMTP_USER	SMTP_PASS
Brevo (free 300/day)	smtp-relay.brevo.c587		the funny xxxxx@smtp-brevo.c587 id from Brevo's SMTP & API page	the SMTP key starting xsmtpsib-...
SendGrid	smtp.sendgrid.net	587	apikey	your SendGrid API key
Gmail (workspace)	smtp.gmail.com	587	your Gmail address	a 16-char app password
Mailgun	smtp.mailgun.org	587	postmaster@mg.yourdomain.com	the Mailgun SMTP password

Critical field: `SMTP_FROM` (relabelled "Display 'From' Address") - must be on a domain your relay has *verified*. The address recipients will see in their inbox. **Not** the SMTP login - those are different values. Common gotcha: pasting the relay's internal id (xxxxx@smtp-brevo.com) into `SMTP_FROM` makes every email get spam- filtered. OSCAR will warn you inline if it spots this pattern.

Click **Save** -> then **Send test email** to yourself. If it arrives, SMTP is working end-to-end. If not, the verbatim relay error is shown on the page (typically `535 authentication failed` or `550 sender`

domain not verified).

Once SMTP works, self-service password reset, registration verification, and admin alerts (Step 8) all work automatically - they share this same config.

DNS hardening (recommended once you're past the test stage): Add SPF + DKIM records for your domain in your DNS zone, following your SMTP relay's instructions. Without them, your emails will work but may land in spam. Brevo, SendGrid, etc. all show you the exact records to add and provide a "Verify" button. This is a one-time setup that benefits every email OSCAR sends.

Step 8 - (Optional) Observability + alerting

If you want live dashboards, centralised logs, and email alerts when OSCAR breaks, bring up the metrics overlay. Otherwise skip this section - the basic install is complete.

```
cd /opt/OSCAR/OSCAR_Deploy

# Add the nginx snippet for /grafana/, /prometheus/, /auth/sso-check
# (paste the contents of nginx/oscar-metrics.conf.snippet inside your
# server { ... } block in /etc/nginx/sites-available/oscar.example.org,
# replace oscar.uic.org references with your domain, then reload)
sudo nginx -t && sudo systemctl reload nginx

# Bring up the full stack
sudo docker compose -f docker-compose.yml -f docker-compose.metrics.yml up -d
```

Then in the admin UI:

1. **Server Config tab -> Alerting card** - fill in **ALERT_RECIPIENTS** (the admin email(s) that should receive alerts). Save.
2. Click **** Apply alerting config to Alertmanager****. OSCAR generates **alertmanager.yml** from the SMTP credentials you set in Step 7 and the recipient list, writes it to a shared docker volume, and hot-reloads Alertmanager.
3. Verify with a synthetic alert: **bash curl -XPOST http://127.0.0.1:9093/api/v2/alerts -H 'Content-Type: application/json' -d ' [{ "labels": { "alertname": "TestAlert", "severity": "critical" }, "annotations": { "summary": "Synthetic test alert" } }]'** # Should email all ALERT_RECIPIENTS within ~1 min
4. Visit **https://oscar.example.org/grafana/** to see the OSCAR - Overview and OSCAR - Logs dashboards.

Full operational reference: **OSCAR - Server Admin Guide.md** §§ 13-14.

Step 9 - (Optional) Disable Watchtower if you want manual upgrades

By default Watchtower auto-pulls every new release. If you'd rather pin to a specific version and upgrade on your own schedule:

```
# Stop Watchtower
sudo docker compose stop watchtower
sudo docker compose rm -f watchtower

# Pin OSCAR to a specific tag in docker-compose.yml - change:
# image: ghcr.io/top-phe/oscar-server:stable
# to:
# image: ghcr.io/top-phe/oscar-server:server-v1.9.0
sudo nano /opt/OSCAR/OSCAR_Deploy/docker-compose.yml
sudo docker compose up -d
```

To upgrade later: bump the tag in compose, `docker compose up -d`, done. The full list of available tags is at github.com/TOP-PHE/UIC_OSCAR-.../pkgs/container/oscar-server.

Day-2 operations

Backup the database

The SQLite DB lives at `/opt/OSCAR/OSCAR_Deploy/data/oscar.db`. A nightly file copy is enough - SQLite is a single file:

```
# Add to root's crontab (`sudo crontab -e`):
0 3 * * * /usr/bin/cp /opt/OSCAR/OSCAR_Deploy/data/oscar.db /opt/OSCAR/backups/oscar-$(date +%F).db
```

For a richer backup that survives mid-write, use SQLite's `.backup` command from inside the container:

```
sudo docker exec oscar sqlite3 /app/data/oscar.db ".backup /app/data/backup.db"
sudo cp /opt/OSCAR/OSCAR_Deploy/data/backup.db /opt/OSCAR/backups/oscar-$(date +%F).db
```

Check what's running

```
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
docker logs oscar --tail 50 # OSCAR application logs
docker logs oscar --since 5m -f # follow live
```

Verify image freshness

```
docker inspect oscar --format '{{.Config.Image}} {{.Image}}'
# Image hash should change after each Watchtower pull
```

Move to a bigger VPS later

Easy because OSCAR is stateless except for the `data/` directory:

```
# On the OLD VPS
sudo docker compose down
sudo tar czf /tmp/oscar-data.tgz -C /opt/OSCAR/OSCAR_Deploy data .env
scp /tmp/oscar-data.tgz user@new-vps:/tmp/

# On the NEW VPS (after Steps 1-4 complete with the SAME .env)
sudo tar xzf /tmp/oscar-data.tgz -C /opt/OSCAR/OSCAR_Deploy
sudo docker compose up -d
# Update DNS A record -> new IP. Done.
```

Troubleshooting top 5

Symptom	Likely cause	Fix
<code>docker pull</code> errors with <code>denied</code> / <code>unauthorized</code>	GHCR package is private	<code>docker logout ghcr.io && docker pull ghcr.io/top-phe/oscar-server:stable</code> . If still denied, the package owner needs to flip it to public.
<code>https://oscar.example.org</code> returns 502 Bad Gateway	OSCAR container down or wrong port	<code>docker logs oscar --tail 50</code> - usually a missing required env var (<code>ENCRYPTION_KEY</code> , <code>JWT_SECRET</code>) caught at startup
Bootstrap call returns 401 / 403	Wrong / missing <code>PLATFORM_BOOTSTRAP_TOKEN</code> in <code>.env</code> , or you forgot to restart oscar after editing <code>.env</code>	<code>sudo docker compose restart oscar</code> , then re-curl
Test email "succeeds" but never arrives	<code>SMTP_FROM</code> on an unverified domain - relay accepts but recipient spam-filters	Use a sender on a domain your relay has verified. Add SPF + DKIM DNS records. Check spam folder.
Server runs but admin UI banner shows "untested combination" in red	<code>compatibility.json</code> doesn't list your image+collection combination	Either upgrade to a known-good combo, or accept the warning - runtime is unaffected

For more, see `metrics-and-monitoring.md` (observability stack issues) and `OSCAR - Server Admin Guide.md` §§ 9 + 14 (general OSCAR ops).

What about contributing back?

The repository is public and accepts contributions:

- **File a bug** - open an issue at the repo's *Issues* tab. No special permissions needed; any GitHub account can create one.
- **Suggest a feature** - same place, label as enhancement.
- **Submit a fix** - fork, branch, PR. CI runs lint + tests + CodeQL + SonarCloud + Gitleaks automatically; once green, a maintainer reviews.

License

OSCAR is published under the Apache License 2.0. You can run, modify, fork, redistribute, and sell access to your own deployment freely. The trademarks "OSCAR" and "OSDM" remain with their respective owners (UIC, the OSDM working group); use them in good faith and only to refer to the project itself.